StellarEdge AI

StellaVLA: In-Context Structured Demonstration for Generalizable Vision-Language-Action Models

StellarEdge AI Technical Team

Date August 1, 2026

https://stelledge.com/blog/stellavla

Abstract

Vision-Language-Action (VLA) models can follow instructions and manipulate objects, but their performance often collapses out of distribution (OOD), when the scene, viewpoint, or object differs from training. Adapting to each new situation typically requires collecting more data and fine-tuning. We present **StellaVLA**, a framework that instead adapts at test time by conditioning on a single retrieved demonstration. The key idea is to move beyond imitating *what* an expert did and instead convey *why*: an automated offline pipeline converts each raw trajectory into a *structured demonstration*, *e.g.*, a task plan, sub-goal descriptions, and verbalized 3D motion, at zero human-annotation cost. Provided as in-context guidance, this structured demonstration lets the policy reason about the task rather than mimic a pixel trajectory, which also makes it transferable across embodiments (real-robot, human-hand, or XR demonstrations). A parallel dual-training design internalizes this reasoning during training through a joint action-and-language objective, while inference uses the action expert alone, preserving real-time, high-frequency control with no added latency. On the VLA-Arena leaderboard[a] (Aug 1, 2026), StellaVLA ranks **first** with an overall score of 0.63, versus 0.44 and 0.22 for the strong prior models ($\pi_{0.5}$ and LingBot-VLA), and it further leads on LIBERO with 98.8% average success rate and LIBERO-Plus with 85.1% success rate. Our real-robot benchmark demonstrates that StellaVLA can use both human/robot demos and human-to-robot (XR) demos as in-context structured demonstration to help VLA model adapt to OOD tasks.

[a] https://vla-arena.github.io/#leaderboard

1 Introduction

Vision-Language-Action (VLA) models have become a prominent paradigm for robotic manipulation [4, 22]. Built on pretrained Vision-Language Models (VLMs), they translate visual and textual inputs into physical actions. In practice, however, their performance degrades sharply out of distribution (OOD), when the scene, viewpoint, or object differs from training [9, 48, 60], and recovering typically requires collecting new data and fine-tuning. In-Context Imitation Learning (ICIL) offers a way to adapt at test time without weight

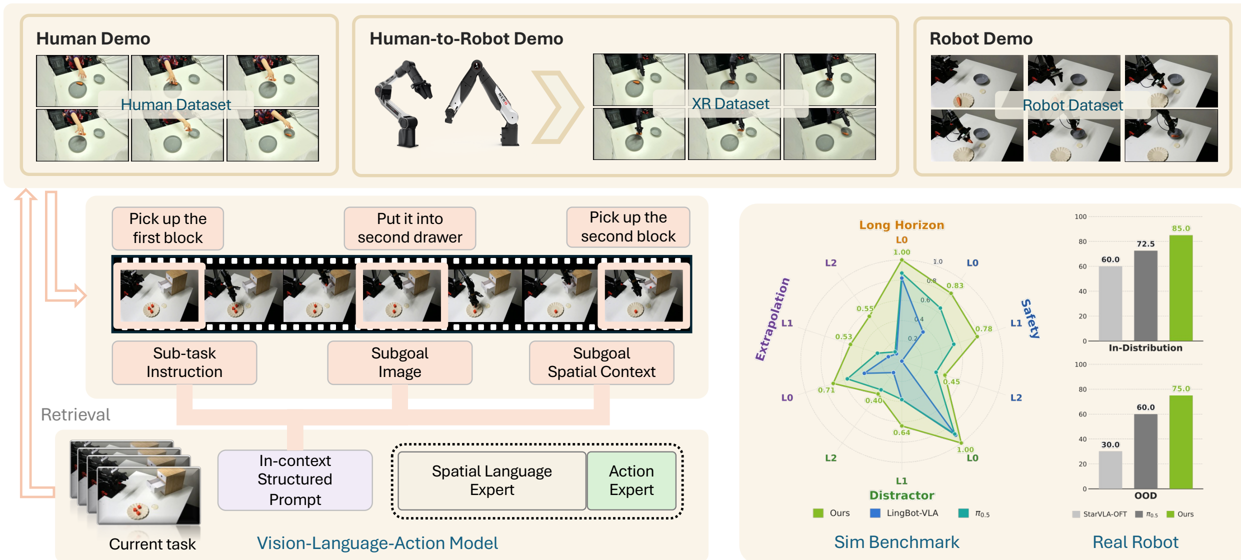

Figure 1 Overview of **StellaVLA**, which conditions a VLA policy on in-context structured demonstrations. We convert diverse human and robot demonstrations into structured examples. Each example comprises high-level *semantic* rationales (sub-goals) and fine-grained *kinematic* rationales (movements), which are retrieved to prompt a vision-language-action model for manipulation tasks. During training, an auxiliary spatial-language expert supervises these rationales so the policy grounds its actions in the underlying reasoning rather than only imitating the observed motions. At inference this expert is removed and only the action expert is used, preserving real-time control. The resulting model generalizes more robustly to unseen tasks and environments in both simulation and real-world experiments.

updates [13, 56]. By appending retrieved expert trajectories, *e.g.*, sequences of observations and low-level actions, as a contextual prefix, ICIL gives the policy a non-parametric memory it can imitate on the fly. But existing frameworks use these trajectories only as raw observations and continuous actions, which encourages surface-level imitation: the policy sees *what* the expert did without the *why*. Lacking that structure, it often treats the demonstration as noise and falls back on its pretrained priors[48], which is a behavioral inertia that keeps it from generalizing to out-of-distribution tasks.

Reasoning provides the missing structure. Foundation models generalize better when the intermediate reasoning is made explicit rather than left implicit [16, 50, 58, 59]: decomposing a task into logical steps exposes the *why* behind each action. The question we address is how to surface this reasoning inside retrieved demonstrations, so that a VLA policy imitates the expert's reasoning rather than only its motions. Motivated by this idea, we propose **StellaVLA** (Figure 1), which advances ICIL by turning demonstrations into structured, reasoning-augmented context. An automated offline pipeline converts raw expert trajectories into this form at zero human-annotation cost: following the "language-as-action" paradigm [8, 28], a VLM segments long-horizon trajectories and generates hierarchical rationales, high-level *semantic* rationales (sub-goals) and fine-grained *kinematic* rationales (movements), which augment each demonstration alongside its original observations and actions. Applied across sources, this yields a unified cross-embodiment demonstration pool.

Unlike prior ICIL methods that rely on prompting alone [20, 57], StellaVLA couples in-context prompting with explicit supervision through a parallel dual-training design. During training, the model conditions on the retrieved rationale-augmented demonstrations and is jointly optimized to predict low-level actions and to articulate the corresponding rationales, internalizing both *what* to do and *why*. The language supervision grounds actions in their semantic intent, while the retrieved rationales act as a reusable template for how to plan in similar situations. At inference, the spatial-language expert is removed entirely: the policy uses only the action expert, guided by the KV-cached demonstration prefix, giving real-time high-frequency control with no added latency while still benefiting from the task structure learned during training.

Our main contributions are:

- We propose **StellaVLA**, a retrieval-augmented VLA framework that shifts ICIL from imitating actions to imitating the reasoning behind them, reducing behavioral inertia under OOD conditions.
- A parallel dual-training design, fed by a zero-human-cost offline extraction pipeline, that internalizes expert reasoning during training and strips the spatial-language expert at inference, leaving the control loop free of autoregressive decoding overhead.
- Extensive simulation and real-robot manipulation experiments show that StellaVLA ranks **first** on the VLA-Arena leaderboard (Aug 1, 2026) with an overall score of **0.63**, while achieving **98.8%** average success in LIBERO and leading in LIBERO-Plus and our real-robot benchmark.

2 Related Work

Vision-language-action (VLA) models adapt pretrained vision-language backbones for robot control. Actions may be generated as discretized tokens [22], or through regression, flow-matching, and frequency-domain heads [4, 23, 34, 36]. Other work improves efficiency [11, 33, 46, 47] or enriches policy representations with geometry, depth, and world knowledge [18, 25, 26, 31, 39, 41, 55]. StellaVLA retains a standard VLM backbone and continuous action expert, and instead studies how demonstrations are represented and used as context.

2.1 Language as an Action Representation

Language-as-action methods express robot behavior in the vocabulary of a pretrained VLM, narrowing the gap between semantic reasoning and continuous control. Prior work renders low-level actions as text, jointly models language and action tokens, or expresses action hierarchies and subtasks in language [2, 15, 27, 51]. Because these representations remain in the VLM's semantic space, they preserve an interpretable connection between task intent and motion and can transfer more naturally across tasks and embodiments.

StellaVLA applies language-as-action to both context and supervision. In the retrieved demonstration, each subgoal combines a semantic description and keyframe with language-rendered robot state and 2D/3D motion, forming a semantic state–action sequence. At the current step, a training-only expert predicts the subtask and a language rendering of the same action chunk targeted by the continuous action expert. The shared language interface connects the segment-level plan to current control without language decoding at inference. In contrast, latent-action methods learn implicit codes outside the VLM's semantic space and require additional alignment or decoding for transfer [5, 49].

2.2 Test-Time Adaptation and Context-Conditioned VLAs

A deployed policy can be adapted either by updating the model or by changing its input context. Test-time training methods optimize fast weights, latent prompts, or memory from deployment data [12, 21, 42, 54]. Conditioning-based methods instead keep the policy fixed and provide demonstrations, retrieved experience, or execution history [30, 32, 38, 56]. Recent methods extend this approach through next-token imitation, post-training, and retrieval-based adaptation [13, 19, 40].

Context can serve different roles. $\pi_{0.7}$ uses subtask instructions, subgoal images, episode metadata, and control mode to specify execution strategy [35], whereas Qwen-RobotManip conditions on recent observation–state–action chunks to adapt to the current episode's execution dynamics [37]. StellaVLA instead retrieves a prior episode and represents it as a task plan abstract with per-segment subtasks and grounded 2D/3D motion.

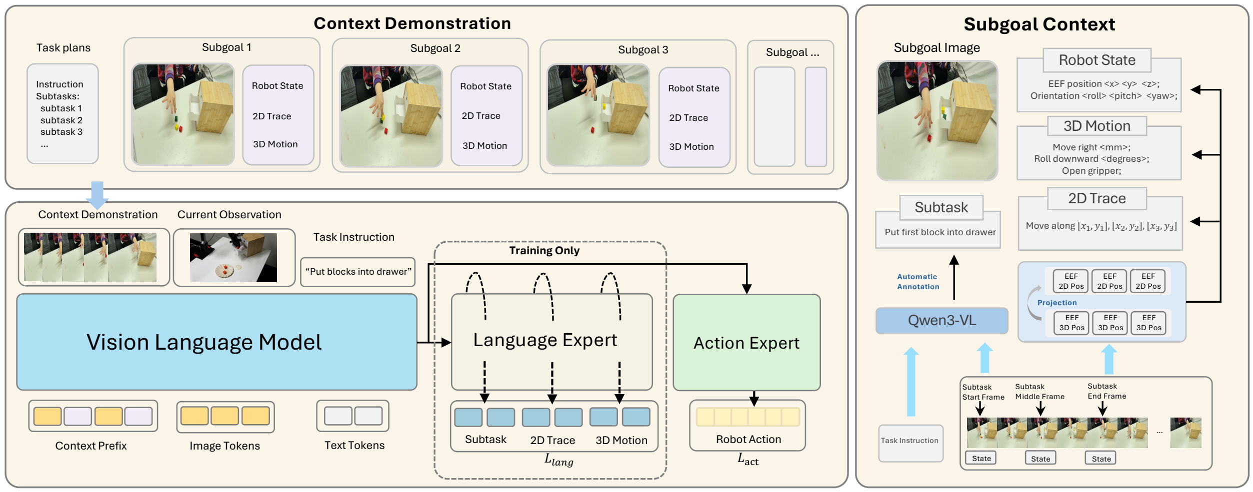

Figure 2 **Overview of StellaVLA.** *Top left:* a retrieved context demonstration is represented as a task plan and a sequence of subgoals, each containing a keyframe, robot state, 2D trace, and 3D motion. *Right:* these annotations are generated automatically offline from VLM reasoning and robot trajectories, without human labeling. *Bottom left:* the vision-language model encodes the demonstration together with the current observation and instruction into a shared representation, which is consumed by an action expert for action prediction and a spatial-language expert for auxiliary supervision during training only. The spatial-language expert is removed at inference, requiring only a single forward pass.

This structured demonstration provides procedural guidance without parameter updates.

2.3 Human and Cross-Embodiment Demonstrations

Human video and heterogeneous robot data offer broader task coverage than single-platform demonstrations, but their observations and action spaces do not align. Existing approaches learn shared latent actions, motion representations, or world models, condition on expert video, or explicitly align embodiments through retargeting and canonical action spaces [3, 6, 7, 14, 17, 37, 43].

StellaVLA takes a representation-based route. Real-robot, human-hand, and XR-retargeted demonstrations are converted into the same structured semantic-and-motion representation before retrieval. Off-embodiment data serves only as context, while executable actions remain supervised in the target robot's control space. This reduces source-specific appearance and coordinate differences before the demonstration reaches the policy.

3 Methodology

In this section, we detail the **StellaVLA** framework, illustrated in Figure 2, which advances ICIL with structured contextual rationales to improve VLA generalization. We first present an automated offline pipeline that converts raw trajectories into rationale-augmented segments (Sec. 3.1), then introduce a parallel dual-training paradigm that uses these trajectories as both explicit supervision and in-context prompts (Sec. 3.2), and finally describe an asymmetric inference strategy that leaves the control loop free of autoregressive decoding overhead (Sec. 3.3).

3.1 Offline Structured Context Extraction

Raw trajectory formulation. An expert demonstration is recorded as $\tau = \{(o_t, a_t)\}_{t=1}^{T}$, where o_t denotes sensory input (*e.g.*, multi-view RGB and proprioception) and $a_t \in \mathbb{R}^{d_a}$ is the continuous control command (*e.g.*, end-effector pose and gripper state). Such trajectories may come from diverse embodiments and collection paradigms, including real-robot teleoperation, human hand tracking, or XR-retargeted demonstrations [43]. Although they encapsulate successful executions, the raw numerical actions lack explicit semantic rationale, which hinders cross-embodiment reasoning.

Semantic segmentation via causal deduction. To bridge the semantic gap without incurring prohibitive human annotation costs, we introduce an automated offline pipeline (Figure 2, right) that deduces the expert's underlying thought process. The core motivation is to infer the "cause" (the expert's underlying structured rationale) from the "effect" (the observed physical execution). Given a raw trajectory τ and its corresponding high-level task instruction $\mathcal{I}$, we utilize a powerful off-the-shelf Vision-Language Model (e.g., Qwen3-VL) to explicitly decompose the continuous trajectory into K discrete, semantically meaningful segments. For each segment $k \in \{1, \ldots, K\}$ spanning from time step $t_{\text{start}}^{(k)}$ to $t_{\text{end}}^{(k)}$, the VLM analyzes the visual changes to identify the high-level sub-goal achieved. This process captures the true decision-making process that governs the expert's behavior.

Kinematic verbalization via language-as-action. Once the trajectory is segmented, we adopt a "language-as-action" paradigm that deterministically translates physical actions within each segment into structured text. Because VLAs are built upon VLMs with a strong affinity for semantic language[15, 53], these rationales can be processed with the original text vocabulary, without introducing specialized action tokens.

Ultimately, this two-tier process generates a **Structured Context** l_k for each segment that explicitly articulates the underlying logic through a hierarchical representation:

- **Semantic Rationale (Sub-goal Description):** Captures the high-level semantic objective achieved in this segment (e.g., *"Reach for the handle of the blue mug"*), automatically identified by the VLM based on visual observations.
- **Kinematic Rationale (Movement Description):** Verbalizes fine-grained motion as a **3D movement** in the workspace (e.g., *"Move the end-effector by* $\Delta x = +0.05, \Delta y = -0.02, \Delta z = +0.10$ *and close gripper"*) and a **2D movement** obtained by projecting the 3D trajectory onto the camera plane with known intrinsics/extrinsics. This dual form encourages the VLM to reason about spatial logic and visual grounding via text tokens, rather than emitting raw floating-point actions.

Both kinematic fields are produced by a single deterministic verbaliser Φ, which maps any contiguous span of actions to its 3D displacement in the workspace and to the 2D projection of that displacement onto the image plane. Applying Φ over a whole segment yields the movement description carried inside l_k; we reuse the *same* operator at the much shorter scale of an action chunk in Sec. 3.2, which is what places the retrieved demonstration and the policy's own prediction in one common vocabulary. The sub-goal description, by contrast, is attached to every frame: each step t carries the subtask s_t that the expert is executing in the observation o_t.

Rationale-augmented demonstration pool. Through this process, the original raw trajectory τ is transformed into a rationale-augmented trajectory $\tau_{\text{rat}} = \{(o_t, a_t, s_t, l_k)\}_{t=1}^{T}$, where each time step t is now grounded not only by its corresponding observation and action, but also by its own subtask label s_t and by the explicit *segment-level* structured rationale l_k of the segment it belongs to. By processing all available expert demonstrations through this pipeline, we construct a comprehensive demonstration pool $\mathcal{D}_{\text{pool}} = \{\tau_{\text{rat}}^{(i)}\}_{i=1}^{N}$.

3.2 Parallel Dual-Training Paradigm

Retrieval-augmented prompt formulation. During training, the learning process is formulated as a leave-one-out retrieval task. To train the policy to imitate a specific target trajectory τ_{tgt} sampled from $\mathcal{D}_{\mathrm{pool}}$, the system queries the remaining pool $\mathcal{D}_{\mathrm{pool}} \setminus \{\tau_{\mathrm{tgt}}\}$ and retrieves the top-1 rationale-augmented expert trajectory, ranked by cosine similarity between the language embeddings of the task instructions. Retrieval is therefore purely linguistic, which is what lets a demonstration recorded on another embodiment be retrieved for a robot episode. The retrieved trajectory is formatted into a contextual prefix prompt $\mathcal{P}_{\mathrm{demo}}$. The final input to the VLA model at time step t is constructed by concatenating the retrieved demonstration, the current task instruction $\mathcal{I}$, and the current observation o_t:

$$x_t = \left[\mathcal{P}_{\mathrm{demo}} \oplus \mathcal{I} \oplus o_t\right] \tag{1}$$

Crucially, because $\mathcal{P}_{\mathrm{demo}}$ contains the rich l_k descriptions from the experts, it acts as an implicit in-context learning (ICL) signal, providing the model with a cognitive template of how to "think" and plan in similar scenarios.

Parallel experts architecture. The core of our VLA policy is a unified Vision-Language Model backbone f_θ equipped with two parallel experts (Figure 2, bottom left). Given the input x_t, the backbone processes the multimodal sequence into a shared latent representation $h_t = f_\theta(x_t)$. This single latent vector is then simultaneously fed into:

1. An **action expert**, a lightweight MLP head that regresses the continuous action chunk $\hat{A}_t = (\hat{a}_t, \ldots, \hat{a}_{t+H-1})$ spanning the next H control steps.
2. A **spatial-language expert**, the native autoregressive LM head, that emits the rationale $\hat{c}_t = (\hat{s}_t, \hat{m}_t)$ of the current step: the subtask $\hat{s}_t$ being executed in o_t, and the verbalised 3D and 2D movement $\hat{m}_t$ of that very same action chunk.

It is vital to note that these two experts operate strictly in parallel. The action prediction does not causally depend on the generated text tokens at time step t. They share the representation h_t and nothing else.

Spatial language supervision. While the retrieved demonstration provides segment-level context, spatial-language supervision is defined at each time step as

$$c_t = \left(s_t,\ \Phi(A_t)\right), \tag{2}$$

where s_t is the subtask label for o_t, $A_t = (a_t, \ldots, a_{t+H-1})$ is the ground-truth action chunk, and $\Phi(A_t)$ verbalises its 3D and 2D movement. The subtask label provides semantic supervision for the current task stage; the 2D movement grounds the prediction in o_t, while the 3D movement provides kinematic supervision in the workspace.

Dual rationale objective. During training, the model is optimized to minimize a joint objective:

$$\mathcal{L} = \mathcal{L}_{\mathrm{act}}(\hat{A}_t, A_t) + \lambda \mathcal{L}_{\mathrm{lang}}(\hat{c}_t, c_t) \tag{3}$$

where $\mathcal{L}_{\mathrm{act}}$ is the regression loss for the continuous control commands (e.g., L_1 loss), $\mathcal{L}_{\mathrm{lang}}$ is the standard autoregressive cross-entropy loss for the structured rationale text generation, and λ is a balancing coefficient.

The two experts provide complementary supervision for the shared representation h_t. Since Φ is deterministic, A_t and $\Phi(A_t)$ describe the same motion in continuous and linguistic forms. Together, the explicit semantic, grounding, and kinematic supervision in c_t and the contextual guidance from $\mathcal{P}_{\mathrm{demo}}$ encourage h_t to connect the demonstrated task structure and current observation with the robot motion required at time t.

3.3 Asymmetric Inference and Caching

Action-only execution. A fundamental limitation of standard Embodied CoT paradigms is the severe latency introduced by autoregressive text generation during inference[16, 24, 50, 58], which often breaks the strict timing requirements of high-frequency robotic control. Our parallel architecture elegantly resolves this bottleneck. Because the action expert and the spatial-language expert read from h_t independently, and the profound physical "mental model" has already been forged into the backbone weights during training via $\mathcal{L}_{\text{lang}}$, the language read-out becomes strictly optional at deployment.

During inference, the autoregressive spatial-language expert is entirely stripped away. The policy executes a single forward pass through the backbone and the lightweight MLP action expert to output the action chunk $\hat{A}_t$. This asymmetric strategy decouples the acquisition of reasoning (paid for during training) from its execution, so continuous control incurs no autoregressive decoding overhead.

Demonstration prefix caching. To further optimize inference efficiency, we exploit the static nature of the retrieved context[47]. For a given novel task, the retrieved rationale-rich demonstration prefix $\mathcal{P}_{\text{demo}}$ is pinned at the beginning of the rollout and remains immutable for the duration of the episode. Consequently, the key-value (KV) cache for the entire prefix is computed exactly once at $t = 1$. For all subsequent time steps, the policy only needs to forward the live suffix (the current observation o_t), drastically reducing the computational footprint. This caching mechanism ensures that carrying a rich, multi-step rationale demonstration incurs virtually zero marginal cost during the high-frequency control loop.

4 Experiments

We evaluate four questions: (i) whether structured demonstrations improve in-distribution performance and generalization under task and visual shifts, (ii) whether the policy uses the retrieved demonstration and which parts of it carry transferable information, (iii) how spatial-language supervision affects the learned representation, and (iv) whether the resulting policy can use cross-embodiment context without sacrificing deployment efficiency.

4.1 Experimental Setup

Model and Training. StellaVLA couples a Qwen3-VL-4B backbone [1] with an OpenVLA-OFT-style MLP action expert [23]. At each step, the model receives third-person and wrist RGB observations, a language rendering of the robot state, the task instruction, and one retrieved same-task structured demonstration. The action expert regresses an action chunk under an L_1 objective. In parallel, the spatial-language expert is trained by cross-entropy (weight $\lambda = 0.3$) to predict the current subtask and the 2D/3D description of the same action chunk. The subtask, 2D movement, and 3D movement provide semantic, visual-grounding, and kinematic supervision, respectively. The spatial-language expert is not decoded at inference.

All models are fully fine-tuned from Qwen3-VL-4B-Instruct for 30k steps with a global batch size of 128. Context-demonstration dropout is 0.0 when a same-task demonstration is always available and 0.5 otherwise. Appendix A gives optimization, retrieval, and platform-specific details.

Protocol and Matched Control. We report task success rate under each benchmark's official protocol. *StarVLA-OFT* [41] is trained with the same backbone, action expert, and data as StellaVLA, but receives neither a retrieved demonstration nor spatial-language supervision. It therefore measures the gain from the complete StellaVLA design rather than either component alone. To isolate the role of context, Table 5 instead intervenes on a fixed StellaVLA checkpoint by supplying the correct, no, or a wrong-task demonstration at evaluation.

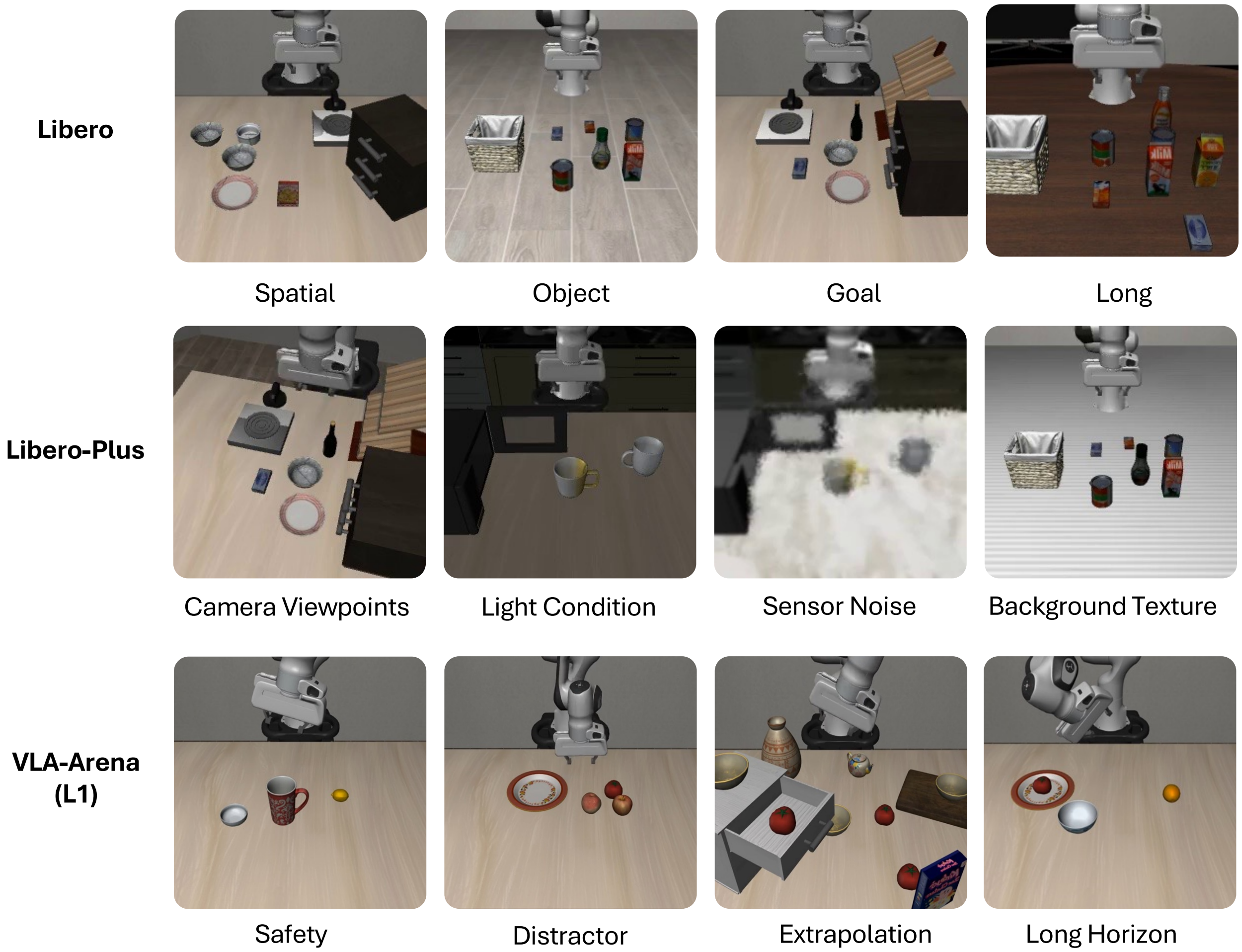

Figure 3 **The three simulation benchmarks.** *Top*: the four standard LIBERO suites. *Middle*: LIBERO-Plus perturbation axes, e.g., camera viewpoint, lighting, sensor noise and background texture. *Bottom*: VLA-Arena, which changes the task, adding safety constraints, distractors, extrapolation to unseen objects, and long-horizon composition.

4.2 Generalization in Simulation

We first evaluate StellaVLA on three simulation benchmarks, as shown in Figure 3, probing in-distribution competence, task-level generalization, and zero-shot robustness.

4.2.1 In-Distribution Validation in LIBERO.

StellaVLA reaches 98.8% average success on LIBERO (Table 1), showing that structured context does not compromise in-distribution performance. Relative to StarVLA-OFT, the gain is small on Object $(+0.4)$ but larger on Goal $(+3.4)$ and Long $(+3.0)$, where the current observation alone may not determine the intended outcome or subgoal order. The same pattern becomes more pronounced when the demonstration is removed or replaced at evaluation (Table 5).

4.2.2 Task-Level Generalization on VLA-Arena.

VLA-Arena [52] evaluates task-level generalization across 11 suites in four categories: Safety, Distractor, Extrapolation, and Long Horizon. Each suite contains three difficulty levels, from L0 (in-distribution) to L2 (hardest), while training uses L0 data only. We follow the official protocol and compare against OpenVLA-

Method	Spatial	Object	Goal	Long	Avg.
MemoryVLA [38]	98.4	98.4	96.4	93.4	96.7
ACoT-VLA [59]	99.4	**99.6**	98.8	96.0	98.5
AVA-VLA [45]	99.2	**99.6**	97.9	96.2	98.2
StarVLA-OFT	97.8	98.6	96.2	93.8	96.6
CogVLA [25]	98.6	98.8	96.6	95.4	97.4
Retrieval-VLA [56]	97.4	98.8	96.3	89.5	95.5
DreamVLA [55]	97.5	94.0	89.5	89.5	92.6
StellaVLA (Ours)	**99.6**	99.0	**99.6**	**96.8**	**98.8**

Table 1 In-distribution success rate (%) on the standard LIBERO benchmark [29]. Each suite is evaluated over 500 rollouts; baseline numbers are as reported in the original papers, except our matched demonstration-free control StarVLA-OFT [41]. Best per column in **bold**.

OFT [23], LingBot-VLA [44], Motus [3], GR00T-N1.6 [31], Evo-Depth [26], and $\pi_{0.5}$ [36]. At test time, StellaVLA receives one structured demonstration of the target task without any parameter update. As shown in Table 2, StellaVLA achieves the best mean success rate at all three levels: 0.84, 0.62, and 0.43 on L0, L1, and L2, respectively. Its overall score is 0.63, compared with 0.44 for the strongest baseline, $\pi_{0.5}$. The margin over $\pi_{0.5}$ grows from 0.15 at L0 to 0.24 at L1 and remains 0.17 at L2, where no parameter update is performed and only the target-task demonstration is supplied.

The per-suite results further delimit this gain. StellaVLA maintains 0.80 on Unseen Objects across all three levels, consistent with transferring a procedure when the object referent changes. Task Workflows rises from 0.64 at L0 to 0.84 at L2, suggesting that context can become more useful as the test workflow departs from training. Long Horizon remains near zero at L1/L2 for every method, including ours: a fixed prefix specifies the procedure but cannot re-plan after execution drift.

4.2.3 Zero-Shot Robustness on LIBERO-Plus.

LIBERO-Plus [10] perturbs LIBERO tasks along seven axes: viewpoint, robot state, sensor noise, object layout, background, lighting, and language. We evaluate the LIBERO-trained checkpoint without retraining and compare with the zero-shot baselines reported in [10]. StellaVLA reaches 85.1% on average (Table 3), outperforming StarVLA-OFT by 10.1 points. The largest gains occur under camera viewpoint ($+23.5$), sensor noise ($+19.7$), robot initial state ($+14.7$), and language perturbations ($+8.3$), where the observation or instruction changes while the task procedure remains intact.

The smaller gains are also informative. Background and lighting are already near saturation for both models, leaving little room for improvement. Under object-layout changes, StellaVLA gains only 0.1 point because the demonstration's scene-specific spatial relation no longer matches the current scene. Structured context therefore transfers task procedure more reliably than a fixed spatial trajectory.

4.3 Real-World and Cross-Embodiment Evaluation

Platform, Tasks, and Data. We evaluate on a 6-DOF AgileX Piper arm with third-person and wrist-mounted RGB cameras. The benchmark contains four tabletop tasks: precision pen-to-cup placement, carrot-to-bowl pick-and-place, placing blocks in a drawer and closing it, and stacking three bowls. We collect 125 teleoperated robot episodes (71,702 frames), covering precision, articulated-object, and multi-stage manipulation.

Cross-Source Demonstration Pool. The retrieval pool combines the robot episodes with 26 human-hand takes recorded in XR and 26 frame-aligned trajectories retargeted to the Piper. All three sources are converted into the same structured representation. The 2D grounding point is aligned across the human wrist and robot

Task Suite	OpenVLA-OFT			LingBot-VLA			Motus			GR00T-N1.6			Evo-Depth			$\pi_{0.5}$			**StellaVLA**		
	L0	L1	L2	L0	L1	L2	L0	L1	L2	L0	L1	L2	L0	L1	L2	L0	L1	L2	L0	L1	L2
Safety																					
Static Obst.	0.89	0.25	0.21	0.47	0.00	0.00	0.82	0.56	0.19	0.72	0.30	0.14	0.88	0.66	0.48	0.90	0.64	0.50	**1.00**	**1.00**	**0.80**
Cautious Grasp	0.67	0.32	0.00	0.21	0.00	0.00	0.76	0.07	**0.19**	0.16	0.02	0.00	0.78	0.24	0.00	0.38	0.16	0.00	**0.80**	**0.50**	0.10
Hazard Avoid.	0.42	0.00	0.09	0.16	0.00	0.00	0.43	0.24	0.16	**0.64**	0.04	0.10	0.40	0.00	0.14	0.62	0.40	0.28	0.62	**0.92**	**0.70**
State Pres.	**0.99**	0.75	0.38	0.54	0.00	0.00	0.85	0.47	0.49	0.66	0.50	0.38	0.88	0.66	0.56	0.80	0.80	**0.74**	0.92	**0.90**	0.62
Dynamic Obst.	0.75	0.55	0.05	0.40	0.00	0.00	0.43	0.35	**0.26**	0.74	0.50	0.02	**0.82**	0.60	0.06	0.54	**0.70**	**0.26**	0.80	0.60	0.02
Distractor																					
Static Distr.	0.99	0.13	0.15	0.93	0.15	0.11	0.75	0.19	0.03	0.46	0.32	0.06	0.94	0.20	**0.24**	0.90	0.04	0.14	**1.00**	**0.56**	0.20
Dynamic Distr.	0.90	0.56	0.39	0.88	0.61	0.17	0.73	0.60	0.33	0.70	**0.72**	0.18	0.86	0.60	0.32	0.88	**0.72**	0.56	**1.00**	**0.72**	**0.60**
Extrapolation																					
Prep. Comb.	0.54	0.09	0.00	0.46	0.05	**0.01**	0.13	0.00	0.00	0.48	0.00	0.00	0.66	0.00	0.00	0.54	0.04	0.00	**0.70**	**0.14**	0.00
Task Workflows	0.53	0.03	0.13	0.37	0.05	0.11	0.32	0.00	0.02	0.42	0.00	0.00	0.32	0.00	0.00	0.52	0.22	0.26	**0.64**	**0.66**	**0.84**
Unseen Obj.	0.61	0.39	0.19	0.34	0.32	0.15	0.59	0.55	0.13	0.26	0.18	0.16	0.78	0.52	0.04	0.64	0.50	0.08	**0.80**	**0.80**	**0.80**
Long Horizon																					
Long Horizon	0.80	0.00	0.00	0.82	0.03	0.00	0.64	**0.05**	**0.03**	0.29	0.02	0.00	0.93	0.00	0.00	0.87	0.00	0.00	**1.00**	0.02	0.00
Mean	0.74	0.28	0.14	0.51	0.11	0.05	0.59	0.28	0.17	0.50	0.24	0.09	0.75	0.32	0.17	0.69	0.38	0.26	**0.84**	**0.62**	**0.43**
Overall	0.39			0.22			0.34			0.28			0.41			0.44			**0.63**		

Table 2 Success rate on VLA-Arena across 3 difficulty levels: L0 (in-distribution), L1 (intermediate generalization), and L2 (hardest). Rows are the 11 task suites grouped into four categories (Safety, Distractor, Extrapolation, Long Horizon). Scores are fractions in $[0, 1]$. Baseline results are from the VLA-Arena leaderboard [52]. Best per column in **bold**.

flange, and off-embodiment frames are used only as retrieved context: all executable action targets remain from the real robot.

Protocol and Baselines. We evaluate four in-distribution tasks, two OOD-L1 object-attribute perturbations, and one unseen OOD-L2 drawer task, with 10 rollouts per cell. Object placements are re-randomized and matched across methods. StarVLA-OFT is the matched control for the complete StellaVLA design, while the separately pretrained $\pi_{0.5}$ [36], fine-tuned on the same episodes, provides an external reference. Appendix B specifies the control stack, perturbations, success criteria, and progress metric.

Results. StellaVLA reaches 85.0% success in distribution and 75.0% on OOD-L1 (Figure 4). On the two tasks with an L1 variant, its average decreases from 80.0% in distribution to 75.0%, a 5-point drop; StarVLA-OFT and $\pi_{0.5}$ drop by 25 and 20 points on the same tasks. This smaller paired degradation indicates greater robustness when the object attributes change, although the StarVLA-OFT comparison measures the demonstration and spatial-language supervision jointly.

No method completes the unseen OOD-L2 task. StellaVLA nevertheless reaches an average progress score of 1.9 out of four, compared with 1.5 for $\pi_{0.5}$ and 1.1 for StarVLA-OFT. This partial progress suggests that the retrieved procedure remains useful, but does not constitute zero-shot task completion.

Demo pair	ID	OOD
$D(\text{real}, \text{none})$	0.0041	0.0045
$D(\text{human}, \text{none})$	0.0041	0.0044
$D(\text{xr}, \text{none})$	0.0041	0.0044
$D(\text{human}, \text{real})$	0.0015	0.0014
$D(\text{xr}, \text{real})$	0.0015	0.0016
$D(\text{human}, \text{xr})$	0.0014	0.0014

Table 4 Normalized action disagreement when only the demonstration source is changed.

Cross-Source Consistency.

The mixed-source rollouts above cannot isolate the effect of demonstration provenance, because each cell samples from all three sources. We therefore evaluate a fixed checkpoint on held-out teleoperated episodes while holding the current observation fixed and changing only the retrieved source. We measure the mean

Method	Orig.	Cam.	Robot	Noise	Layout	Backg.	Light	Lang.	Avg.
OpenVLA	76.5	1.1	4.1	19.3	31.6	25.3	4.4	26.8	16.0
OpenVLA-OFT	97.1	59.7	37.2	76.7	77.1	92.4	85.8	81.5	71.4
π_0	94.2	15.8	6.6	79.4	70.4	78.5	79.6	61.0	53.8
π_0-FAST	85.5	66.4	24.8	75.8	70.3	67.7	73.0	63.3	62.5
Nora	87.9	4.0	41.1	17.6	63.9	50.5	31.0	67.0	38.7
WorldVLA	79.1	0.3	30.2	12.2	39.4	14.5	29.4	44.2	24.3
UniVLA	95.2	4.3	50.3	25.3	34.3	80.0	59.1	71.8	44.0
RIPT-VLA	97.5	58.3	36.7	73.8	76.5	90.4	87.9	80.1	70.4
StarVLA-OFT	96.6	47.0	60.1	73.1	79.2	**95.3**	**96.3**	87.0	75.0
StellaVLA	**98.8**	**70.5**	**74.8**	**92.8**	**79.3**	95.2	95.7	**95.3**	**85.1**

Table 3 Zero-shot robustness on LIBERO-Plus [10]; all models are trained on standard LIBERO and tested on the perturbed tasks without retraining. *Avg.* is the *task-count-weighted* mean over the seven perturbation categories, excluding *Orig.* Best per column in **bold**.

L_1 disagreement $D(S_1, S_2)$ between predicted action chunks, normalized by the per-dimension standard deviation σ of the ground-truth actions. The paired human and XR-retargeted demonstrations share the same trajectory and timing, so their comparison changes embodiment appearance without changing the demonstrated behavior.

Across $1{,}780$ ID and $1{,}719$ OOD frames, Table 4 shows that real-robot, human-hand, and XR-retargeted context yield closely matched predictions. Source-to-source disagreement is 0.0014–0.0016σ, corresponding to $0.02°$–$0.03°$ of joint angle and less than 0.05 mm of gripper width. This supports consistency across the three structured demonstration sources.

This test is limited to single-step predictions. Removing the demonstration changes the action by only 0.0041–0.0045σ, and small differences may still accumulate in closed-loop execution. The result therefore establishes source consistency under matched observations, not closed-loop invariance or the standalone contribution of context on hardware.

4.4 Understanding Structured Demonstrations

Unless stated otherwise, these analyses use the LIBERO checkpoint and report the four-suite average (AVG) and, where available, the LIBERO-Plus average (LP). Evaluation-time interventions change only the demonstration supplied to a fixed checkpoint.

Causal Role of the Demonstration. Table 5 evaluates the same trained checkpoint with the correct demonstration, no demonstration, and a demonstration from a different task. Removing the demonstration reduces the average success rate from 98.8 to 62.4, while providing a wrong-task demonstration further lowers it to 44.9. The latter result is particularly informative: if the policy largely ignored the retrieved context, missing and mismatched demonstrations would produce similar outcomes. Instead, the additional degradation shows that StellaVLA actively uses the demonstration to determine the intended task and adjusts its behavior accordingly, even when the supplied context is misleading.

Demo	Sp.	Obj.	Goal	Long	AVG
Correct	99.6	99.0	99.6	96.8	98.8
None	72.2	94.4	24.8	58.2	62.4
Wrong	72.6	52.0	0.0	55.0	44.9

Table 5 Changing demonstration content at evaluation.

The per-suite results further clarify when this contextual specification matters. On Goal, performance falls to 24.8 without a demonstration and to 0.0 with a wrong one, since tasks may involve similar objects and scenes but require different outcomes. The demonstration is therefore critical for disambiguating the intended goal.

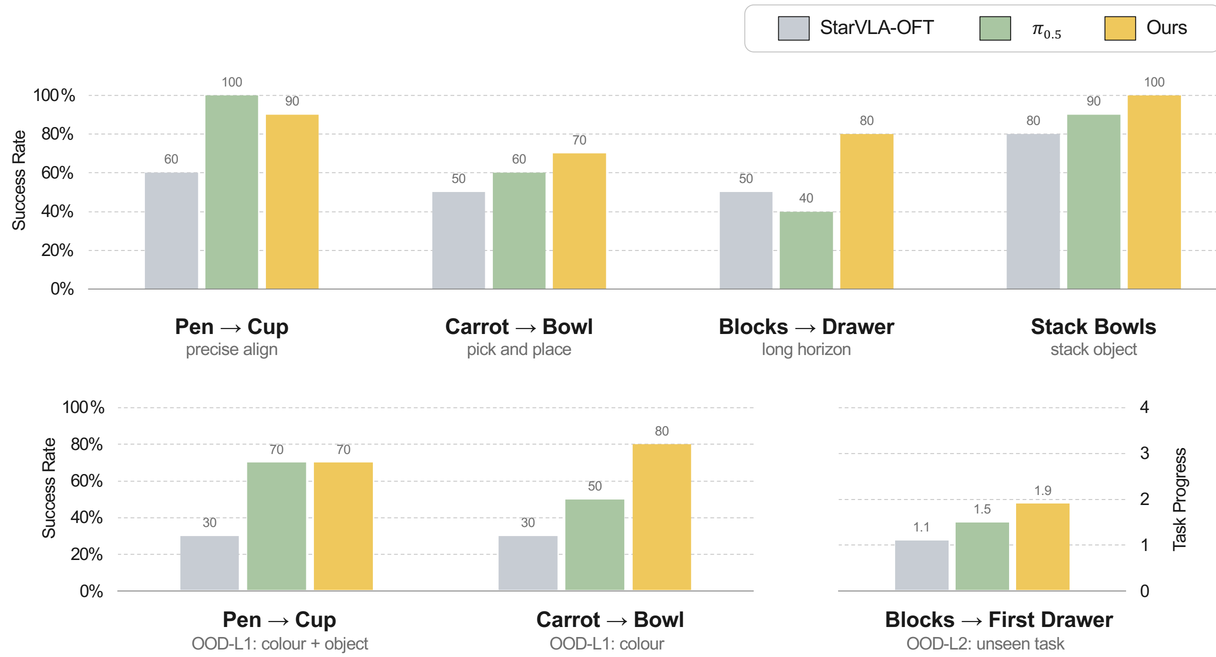

Figure 4 Real-robot evaluation on the AgileX Piper. *Top*: in-distribution success rate on the four tasks. *Bottom left*: *OOD-L1*, which perturbs object attributes and is comparable within a task rather than across tasks. *Bottom right*: *OOD-L2*, an unseen task placing the blocks in the first drawer. Scenes are shown in Figure 5.

By contrast, Spatial performs similarly with no and wrong demonstrations (72.2 and 72.6), suggesting that the current observation still provides useful cues about where the interaction should occur. These results provide direct evidence that the retrieved demonstration serves as a task specification rather than being ignored as auxiliary visual context.

Spatial-Language Components. We next isolate the two movement fields in the structured demonstration. Removing the 3D movement lowers AVG from 98.8 to 97.8, whereas removing the 2D gripper path lowers it to 97.3. Although the 2D path is a projection of the same physical motion, it directly anchors that motion in the current image. Its larger effect is consistent with the 2D component providing visual grounding and the 3D component providing workspace-level kinematic information. This experiment does not isolate the semantic subtask label, for which no removal result is available.

Demonstration Modality. Table 7 examines which part of the demonstration provides the useful context. At evaluation time, text-only demonstrations nearly match the full image+text input, achieving 98.8/84.4 compared with 98.8/85.1 on LIBERO AVG and LIBERO-Plus. In contrast, image-only demonstrations decrease performance to 92.9/75.7, with the largest loss appearing on Long. These results indicate that most of the transferable task information is captured by the structured language representation, including the subgoal sequence and spatial movement, rather than by raw demonstration frames alone. This also supports the use of demonstrations across embodiments, since the structured description abstracts away much of the source-specific visual appearance.

The training-time block of Table 7 reveals a complementary trade-off. Image-only demonstrations provide slightly higher in-distribution performance than text-only demonstrations (98.4 vs. 97.3), but generalize substantially worse on LIBERO-Plus (78.7 vs. 85.0). A plausible explanation is that visual demonstrations allow the policy to exploit appearance correspondence between the demonstration and the current observation, which is effective in distribution but becomes unreliable under visual perturbations. Structured language

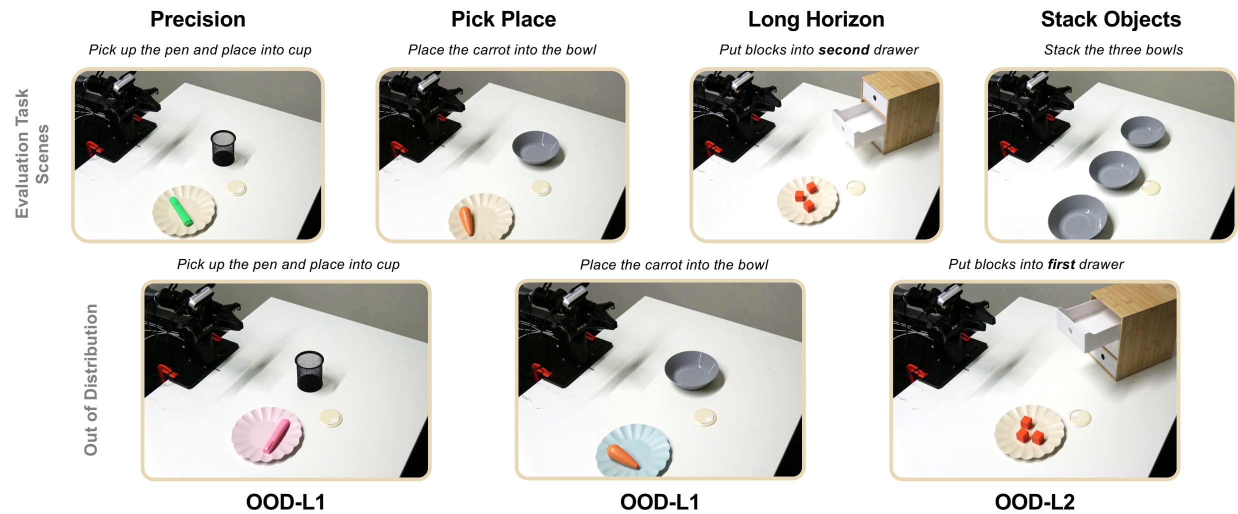

Figure 5 Real-robot evaluation scenes. *Top*: the four in-distribution tasks. *Bottom*: their out-of-distribution variants. *OOD-L1* substitutes both colour and object identity for pen→cup but colour alone for carrot→bowl, which is why L1 is comparable within a task rather than across tasks; *OOD-L2* redirects the blocks from the second drawer to the first, a task for which no real-robot training episode exists.

Demo modality	Sp.	Obj.	Goal	Long	AVG	LP
Changed at evaluation (fixed checkpoint)						
Image+Text	99.6	99.0	99.6	96.8	**98.8**	**85.1**
Text-only	99.0	99.8	99.8	96.6	**98.8**	84.4
Image-only	96.4	98.6	96.0	80.4	92.9	75.7
Changed at training						
Text-only	98.6	98.6	99.2	92.8	97.3	**85.0**
Image-only	99.4	99.4	98.8	95.8	**98.4**	78.7

Table 7 Demonstration modality, changed at evaluation on a fixed checkpoint (top) and at training (bottom). Image+Text is the default. Best per column within each block in **bold**.

removes much of this shortcut and instead encourages the policy to rely on task-level and spatial structure that remains more stable across distribution shifts.

Context Granularity. Performance changes by only 0.7 point when the demonstration is reduced from ten to three subgoal keyframes (Table 9 in Appendix C). This saturation suggests that the subgoal sequence, rather than dense trajectory replay, carries the useful context.

λ	AVG	LP
0	97.6	**86.9**
0.3	**98.8**	85.1
0.6	97.5	84.0
1.0	97.2	81.9

Table 6 Spatial-language loss weight.

4.5 Spatial-Language Supervision and Deployment

Effect of Language-Loss Weight. Table 6 varies the weight λ of the spatial-language objective. The in-distribution average follows a shallow inverted-U and peaks at $\lambda = 0.3$ (98.8), whereas LIBERO-Plus robustness decreases from 86.9 at $\lambda = 0$ to 81.9 at $\lambda = 1.0$. Spatial-language supervision therefore improves in-distribution precision at the selected weight but does not monotonically improve OOD robustness; context conditioning alone already provides a strong robustness signal. One possible explanation is that a large language weight overemphasizes the offline annotation schema. We use $\lambda = 0.3$ as the best in-distribution setting.

Inference Efficiency. Table 8 separates the cost of context encoding from language decoding. A forward pass without a demonstration takes 64 ms. Adding an image+text demonstration raises this to 183 ms without a cache, while caching the immutable prefix reduces the steady-state cost to 91 ms. In a paired measurement, the action-only path takes 88 ms, whereas decoding the 83-token spatial-language output increases latency to 3177 ms, about 36× slower.

Configuration	Cache	Latency
No demo	—	64
Image+Text	no	183
Image+Text	yes	91
Text-only	—	109
Paired language decoding		
Action only	yes	88
+ language	yes	3177

Table 8 Inference latency (ms).

The 88–91 ms values measure the model-side path; the real-robot pipeline, including observation and control overhead, runs at approximately 205 ms per action chunk (Appendix B). Thus, removing the spatial-language expert avoids autoregressive decoding at deployment, while prefix caching amortizes demonstration encoding across the rollout.

5 Conclusion

We presented **StellaVLA**, a framework that improves VLA generalization by converting retrieved expert demonstrations into structured in-context guidance. An automated offline pipeline extracts hierarchical semantic and kinematic rationales from heterogeneous demonstrations, while a parallel dual-training objective jointly learns continuous control and spatial-language reasoning. At inference, the spatial-language expert is removed and the fixed demonstration prefix is KV-cached, allowing the policy to benefit from structured reasoning without autoregressive language-decoding overhead. Experiments on LIBERO, VLA-Arena, LIBERO-Plus, and real-world manipulation show that StellaVLA preserves strong in-distribution performance while substantially improving robustness to task and environment shifts, including when demonstrations originate from different embodiments.

6 Author List

Contributor: Siyu Xu, Yunke Wang, Zijian Wang, Dihao Zhu, Chenghao Xia, Chengbin Du, Daochang Liu, Tao Huang, Chang Xu.

References

[1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, et al. Qwen3-VL Technical Report. *arXiv preprint arXiv:2511.21631*, 2025.

[2] Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debidatta Dwibedi, and Dorsa Sadigh. Rt-h: Action hierarchies using language. In *https://arxiv.org/abs/2403.01823*, 2024.

[3] Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, Haitian Liu, Ruowen Zhao, Yao Feng, Chendong Xiang, Yinze Rong, Hongyan Zhao, Hanyu Liu, Zhizhong Su, Lei Ma, Hang Su, and Jun Zhu. Motus: A Unified Latent Action World Model. *arXiv preprint arXiv:2512.13030*, 2025.

[4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164*, 2024.

[5] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Univla: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025.

[6] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. UniVLA: Learning to Act Anywhere with Task-Centric Latent Actions. *arXiv preprint arXiv:2505.06111*, 2025.

[7] Guangyan Chen, Meiling Wang, Qi Shao, Zichen Zhou, Weixin Mao, Te Cui, Minzhao Zhu, Yinan Deng, Luojie Yang, Zhanqi Zhang, Yi Yang, Hua Chen, and Yufeng Yue. See Once, Then Act: Vision-Language-Action Model with Task Learning from One-Shot Video Demonstrations. *arXiv preprint arXiv:2512.07582*, 2025.

[8] William Chen, Jagdeep Singh Bhatia, Catherine Glossop, Nikhil Mathihalli, Ria Doshi, Andy Tang, Danny Driess, Karl Pertsch, and Sergey Levine. Steerable Vision-Language-Action Policies for Embodied Reasoning and Hierarchical Control. *arXiv preprint arXiv:2602.13193*, 2026.

[9] Irving Fang, Juexiao Zhang, Shengbang Tong, and Chen Feng. From intention to execution: Probing the generalization boundaries of vision-language-action models. *arXiv preprint arXiv:2506.09930*, 2025.

[10] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. LIBERO-Plus: In-depth Robustness Analysis of Vision-Language-Action Models. *arXiv preprint arXiv:2510.13626*, 2025.

[11] Yixu Feng, Zinan Zhao, Yanxiang Ma, Chenghao Xia, Chengbin Du, Yunke Wang, and Chang Xu. See what matters: Differentiable grid sample pruning for generalizable vision-language-action model. In *Forty-third International Conference on Machine Learning*, 2026.

[12] Yusen Feng, Bingchen Han, Jiangran Lyu, Kai Liu, Yixin Zheng, Yuxuan Wan, Weiheng Liu, Sun Han, Ruiqin Li, Yulong Zhang, Fangfu Liu, Xuesong Shi, Libin Liu, Yizhou Wang, Zhizheng Zhang, and He Wang. WAM-TTT: Steering World-Action Models by Watching Human Play at Test Time. *arXiv preprint arXiv:2607.06988*, 2026.

[13] Letian Fu, Huang Huang, Gaurav Datta, Lawrence Yunliang Chen, William Chung-Ho Panitch, Fangchen Liu, Hui Li, and Ken Goldberg. Icrt: In-context imitation learning via next-token prediction. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

[14] Shenyuan Gao, William Liang, Kaiyuan Zheng, Seonghyeon Ye, Qianli Ma, Ruijie Zheng, Pieter Abbeel, Yuke Zhu, Joel Jang, Linxi Fan, et al. DreamDojo: A Generalist Robot World Model from Large-Scale Human Videos. *arXiv preprint arXiv:2602.06949*, 2026.

[15] Asher J. Hancock, Xindi Wu, Lihan Zha, Olga Russakovsky, and Anirudha Majumdar. Actions as Language: Fine-Tuning VLMs into VLAs Without Catastrophic Forgetting. *arXiv preprint arXiv:2509.22195*, 2025.

[16] Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Frank Wang, and Fu-En Yang. Thinkact: Vision-language-action reasoning via reinforced visual latent planning. *Advances in Neural Information Processing Systems*, 38: 82782–82802, 2026.

[17] Chenyu Hui, Xiaodi Huang, Siyu Xu, Yunke Wang, Shan You, Fei Wang, Tao Huang, and Chang Xu. Seeing realism from simulation: Efficient video transfer for vision-language-action data augmentation. *arXiv preprint arXiv:2605.02757*, 2026.

[18] Chia-Yu Hung, Qi Sun, Pengfei Hong, Amir Zadeh, Chuan Li, U.-Xuan Tan, Navonil Majumder, and Soujanya Poria. NORA: A Small Open-Sourced Generalist Vision Language Action Model for Embodied Tasks. *arXiv preprint arXiv:2504.19854*, 2025.

[19] Sanghwan Jang, Minjin Jeon, Minsoo Kim, Seong Jin Choi, Dongha Kim, and Hwanjo Yu. Ra-vla: Retrieval-augmented vla for test-time adaptation. In *Proceedings of the 43rd International Conference on Machine Learning*. PMLR, 2026. URL https://openreview.net/forum?id=ut6HebnnQe.

[20] Sanghwan Jang, Minjin Jeon, Minsoo Kim, Seong Jin Choi, Dongha Kim, and Hwanjo Yu. Ra-vla: Retrieval-augmented vla for test-time adaptation. In *Forty-third International Conference on Machine Learning*, 2026.

[21] Yunfan Jiang, Yevgen Chebotar, Ruijie Zheng, Fengyuan Hu, Yunhao Ge, Jimmy Wu, Tianyuan Dai, Scott Reed, Li Fei-Fei, and Yuke Zhu. RoboTTT: Context Scaling for Robot Policies. *arXiv preprint*, 2026.

[22] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv preprint arXiv:2406.09246*, 2024.

[23] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success. *arXiv preprint arXiv:2502.19645*, 2025.

[24] Jason Lee, Jiafei Duan, Haoquan Fang, Yuquan Deng, Shuo Liu, Boyang Li, Bohan Fang, Jieyu Zhang, Yi Ru Wang, Sangho Lee, et al. Molmoact: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025.

[25] Wei Li, Renshan Zhang, Rui Shao, Jie He, and Liqiang Nie. CogVLA: Cognition-Aligned Vision-Language-Action Model via Instruction-Driven Routing & Sparsification. *arXiv preprint arXiv:2508.21046*, 2025.

[26] Tao Lin, Yuxin Du, Jiting Liu, Nuobei Zhu, Yunhe Li, Yuqian Fu, Yinxinyu Chen, Hongyi Cai, Zewei Ye, Bing Cheng, Kai Ye, Yiran Mao, Yilei Zhong, MingKang Dong, Junchi Yan, Gen Li, and Bo Zhao. Evo-Depth: A Lightweight Depth-Enhanced Vision-Language-Action Model. *arXiv preprint arXiv:2605.14950*, 2026.

[27] Tao Lin, Yuxin Du, Yiran Mao, Zewei Ye, Yilei Zhong, Bing Cheng, Yiming Wang, Jiting Liu, Yang Tian, Junchi Yan, Feiran Wu, Zenan Meng, Hu Wei, Yuqian Fu, Gen Li, and Bo Zhao. LA4VLA: Learning to Act without Seeing via Language-Action Pretraining. *arXiv preprint arXiv:2606.27295*, 2026.

[28] Tao Lin, Yuxin Du, Yiran Mao, Zewei Ye, Yilei Zhong, Bing Cheng, Yiming Wang, Jiting Liu, Yang Tian, Junchi Yan, et al. La4vla: Learning to act without seeing via language-action pretraining. *arXiv preprint arXiv:2606.27295*, 2026.

[29] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. *arXiv preprint arXiv:2306.03310*, 2023.

[30] LocoFormer. LocoFormer: Generalist Locomotion via Long-Context Adaptation. OpenReview, 2025.

[31] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Jim Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. *arXiv preprint arXiv:2503.14734*, 2025.

[32] Austin Patel, Ben Pekarek, Joel Enrique Castro Hernandez, and Shuran Song. Behavior Prompting Policy: Demonstrations as Prompts for Manipulation. *arXiv preprint arXiv:2606.30457*, 2026.

[33] Xiaohuan Pei, Yuxing Chen, Siyu Xu, Yunke Wang, Yuheng Shi, and Chang Xu. Action-aware dynamic pruning for efficient vision-language-action manipulation. In *International Conference on Learning Representations*, volume 2026, pages 10832–10851, 2026.

[34] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. FAST: Efficient Action Tokenization for Vision-Language-Action Models. *arXiv preprint arXiv:2501.09747*, 2025.

[35] Physical Intelligence. $\pi_{0.7}$: A Steerable Generalist Robotic Foundation Model with Emergent Capabilities. *arXiv preprint arXiv:2604.15483*, 2026.

[36] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[37] Qwen Team. Qwen-RobotManip Technical Report: Alignment Unlocks Scale for Robotic Manipulation Foundation Models. *arXiv preprint arXiv:2606.17846*, 2026.

[38] Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. MemoryVLA: Perceptual-Cognitive Memory in Vision-Language-Action Models for Robotic Manipulation. *arXiv preprint arXiv:2508.19236*, 2026.

[39] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. *arXiv preprint arXiv:2506.01844*, 2025.

[40] Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. Ricl: Adding in-context adaptability to pre-trained vision-language-action models. *arXiv preprint arXiv:2508.02062*, 2025.

[41] StarVLA Community. StarVLA: A Lego-like Codebase for Vision-Language-Action Model Developing. *arXiv preprint arXiv:2604.05014*, 2026.

[42] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. *arXiv preprint arXiv:1909.13231*, 2020.

[43] Jen-Wei Wang, Sarthak Kaingade, Andrea Tagliabue, and Nicholas Morozovsky. X-op: Cross-morphology whole-body teleoperation via mpc retargeting. *arXiv preprint arXiv:2606.07934*, 2026.

[44] Wei Wu, Fangjing Wang, Fan Lu, He Sun, Shi Liu, Yunnan Wang, Yibin Yan, Yong Wang, Shuailei Ma, Xinyang Wang, Yibin Liu, Shuai Yang, Tianxiang Zhou, Kejia Zhang, Lei Zhou, Cheng Su, Nan Xue, Bin Tan, Han Zhang, Youchao Zhang, Fei Liao, Xing Zhu, Yujun Shen, and Kecheng Zheng. From Foundation to Application: Improving VLA Models in Practice. *arXiv preprint arXiv:2607.06403*, 2026.

[45] Lei Xiao, Jifeng Li, Juntao Gao, Feiyang Ye, Yan Jin, Jingjing Qian, Jing Zhang, Yong Wu, and Xiaoyuan Yu. AVA-VLA: Improving Vision-Language-Action Models with Active Visual Attention. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13453–13463, 2026.

[46] Weiying Xie, Qingchen Zeng, Zihan Meng, Jiayun Tian, Sibo He, Danian Yang, Jie Du, Yunke Wang, Daixun Li, Hengyi Wang, et al. Towards efficient embodied reasoning: Mixture-of-depth compute allocation for vision-language-action model. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 5732–5741, 2026.

[47] Siyu Xu, Yunke Wang, Chenghao Xia, Dihao Zhu, Tao Huang, and Chang Xu. Vla-cache: Efficient vision-language-action manipulation via adaptive token caching. *Advances in Neural Information Processing Systems*, 38:164448–164473, 2026.

[48] Siyu Xu, Zijian Wang, Yunke Wang, Chenghao Xia, Tao Huang, and Chang Xu. Affordance field intervention: Enabling vlas to escape memory traps in robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 37206–37215, 2026.

[49] Seonghyeon Ye, Joel Jang, Byeongguk Jeon, Sejune Joo, Jianwei Yang, Baolin Peng, Ajay Mandlekar, Reuben Tan, Yu-Wei Chao, Bill Yuchen Lin, Lars Liden, Kimin Lee, Jianfeng Gao, Luke Zettlemoyer, Dieter Fox, and Minjoon Seo. Latent Action Pretraining from Videos. *arXiv preprint arXiv:2410.11758*, 2024.

[50] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. *arXiv preprint arXiv:2407.08693*, 2024.

[51] Lihan Zha, Asher J. Hancock, Mingtong Zhang, Tenny Yin, Yixuan Huang, Dhruv Shah, Allen Z. Ren, and Anirudha Majumdar. LAP: Language-Action Pre-Training Enables Zero-Shot Cross-Embodiment Transfer. *arXiv preprint arXiv:2602.10556*, 2026.

[52] Borong Zhang, Jiahao Li, Jiachen Shen, Yishuai Cai, Yuhao Zhang, Yuanpei Chen, Juntao Dai, Jiaming Ji, and Yaodong Yang. VLA-Arena: An Open-Source Framework for Benchmarking Vision-Language-Action Models. *arXiv preprint arXiv:2512.22539*, 2025.

[53] Fengnian Zhang, Tao Huang, Siyu Xu, Zhong Jin, and Chang Xu. Revisiting parameter redundancy in vision-language-action models: Insights from vlm-to-vla adaptation. *arXiv preprint arXiv:2606.31382*, 2026.

[54] Wenbo Zhang, Jianxiong Li, Shuai Yang, Sijin Chen, Jiajun Liu, Lingqiao Liu, and Xiao Ma. TTT-VLA: Test-Time Latent Prompt Optimization for Vision-Language-Action Models. *arXiv preprint arXiv:2606.03127*, 2026.

[55] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xinqiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, Fan Lu, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A Vision-Language-Action Model Dreamed with Comprehensive World Knowledge. *arXiv preprint arXiv:2507.04447*, 2025.

[56] Yue Zhang, Rui Wang, Jiehong Lin, Zhongrui Wang, and Xiaojuan Qi. Retrieval-VLA: Training-Free In-Context Adaptation for Vision-Language-Action Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1358–1367, 2026.

[57] Yue Zhang, Rui Wang, Jiehong Lin, Zhongrui Wang, and Xiaojuan Qi. Retrieval-vla: Training-free in-context adaptation for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1358–1367, 2026.

[58] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Ming-Yu Liu, Donglai Xiang, Gordon Wetzstein, and Tsung-Yi Lin. CoT-VLA: Visual Chain-of-Thought Reasoning for Vision-Language-Action Models. *arXiv preprint arXiv:2503.22020*, 2025.

[59] Linqing Zhong, Yi Liu, Yifei Wei, Ziyu Xiong, Si Liu, and Guanghui Ren. ACoT-VLA: Action Chain-of-Thought for Vision-Language-Action Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8152–8162, 2026.

[60] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.

A Implementation Details

Optimization. All models are fully fine-tuned from Qwen3-VL-4B-Instruct with AdamW ($\beta = 0.9/0.95$, weight decay 10^{-8}), a backbone learning rate of 5×10^{-6} and an action-expert learning rate of 10^{-4} under a cosine schedule (500 warmup steps), gradient clipping 1.0, bf16 mixed precision, and DeepSpeed ZeRO-2, for 30k steps at a global batch size of 128.

Regularization. Two per-sample dropouts regularize training. A 2D gripper-path dropout (0.5) keeps the 3D steering command usable when the pixel trace is missing. A context-demonstration dropout matches demonstration availability at deployment: 0.0 when a same-task demonstration is always retrievable (LIBERO, LIBERO-Plus) and 0.5 otherwise (VLA-Arena, the real robot).

Retrieval. Demonstrations are retrieved by the language embedding of the task instruction alone; no visual features enter the query. In simulation the instruction set is closed, so this reduces to an exact task match, and we condition on the single nearest demonstration ($M = 1$) throughout. Training uses leave-one-out retrieval, which excludes the target episode from the pool.

Action Expert and Spatial-Language Expert. A set of action-placeholder tokens is appended to the user turn, causally preceding the language-action chain-of-thought. After a single backbone forward, their final-layer hidden states pass through a two-block residual MLP that regresses the action chunk under an L_1 objective. The compact "subtask + 3D/2D steering command" chain-of-thought is retained from a language-action formulation [51] and supervised against offline-generated annotations by cross-entropy; it is not decoded at action inference. The steering command is the target $\Phi(A_t)$ of Eq. 2, that is, the same action chunk the MLP regresses, rendered by the verbaliser Φ over the chunk span rather than over a whole segment; the subtask is the per-frame label s_t.

Per-Platform Instantiation. In simulation the policy reads 256×256 images, predicts end-effector deltas over an action chunk of length 8, and the retrieved demonstration carries up to ten subgoal keyframes. On the real robot it reads 640×480 images, predicts a 7-dimensional chunk of length 16 (six joint-position deltas relative to the episode start plus an absolute gripper width, all q_{99}-normalized), and the demonstration carries up to eight keyframes. The real-robot state is rendered in language as the end-effector pose together with the joint angles.

B Real-World Setup

Control Stack. The AgileX Piper is driven over CAN at 30 Hz without a ROS stack. Each 16-step chunk spans $\approx$0.53 s and is executed in a synchronous receding-horizon loop with a horizon of 8. Joint targets pass through a One Euro filter and a $25°$ per-step jump guard before reaching the arm. Because the demonstration prefix is fixed for the duration of a rollout, its key–value cache is computed once and reused, and steady-state inference settles at $\approx$205 ms per chunk, comfortably inside the control budget.

Data Collection. The 125 teleoperated episodes (71,702 frames) were collected by master–slave teleoperation across the four tasks: *pick the pen and put it in the cup*, *pick the carrot and put it in the bowl*, *put the blocks into the second drawer and close it*, and *stack the three bowls on top of each other*. They cover precision alignment, free-space pick-and-place, a long-horizon sequence involving an articulated object, and multi-stage stacking respectively.

Cross-Source Pool Construction. Twenty-six human-hand takes recorded in XR yield two aligned datasets of 26 episodes and 10,996 frames each: one preserving the operator's bare hand, one showing a retargeted Piper arm executing the identical trajectory. The two share episode structure and timing frame for frame, so the appearance of the embodiment is the only variable separating them. The grounding point of the 2D gripper path uses the operator's wrist for the human take and the flange for the retargeted arm, both matched to the real robot's convention, so that a rendered coordinate denotes the same physical quantity regardless of provenance. The three sources are merged by exact task-string match, retrieval excludes the current episode, and the off-embodiment data enters *only* the retrieval pool: no human or retargeted frame ever supplies an action target, so the policy's motor supervision remains entirely real-robot.

Evaluation Conditions. Every reported cell is 10 rollouts scored by a human operator against a fixed success criterion, with object placements re-randomized per rollout and held identical across methods. OOD-L1 severity is not calibrated across tasks: it compounds a colour and an object substitution for pen→cup but changes only colour for carrot→bowl, so L1 is comparable within a task rather than across tasks. On OOD-L2 the progress score awards one point for each of the three blocks placed and one for closing the drawer, out of four. Single cells carry roughly 10–15 points of noise at 10 rollouts, so we base conclusions on the per-condition averages, which aggregate 40 rollouts in distribution and 20 under L1, and on paired degradations. Because only pen→cup and carrot→bowl have an L1 variant, the degradation quoted in the main text compares each policy against its own score *on those two tasks* (80.0% for StellaVLA, 55.0% for StarVLA-OFT, 80.0% for $\pi_{0.5}$) rather than against the four-task in-distribution average. On OOD-L2 the progress scores are 1.9 for StellaVLA, 1.5 for $\pi_{0.5}$ and 1.1 for StarVLA-OFT; the task is absent from real-robot training, so the human take is the only entry in the pool holding a demonstration of it.

Baselines. The matched control is *StarVLA-OFT* [41], trained on the same real-robot episodes with the same backbone, action expert and action space, and differing only in receiving no demonstration and no auxiliary spatial-language expert. We additionally fine-tune the openpi flow-matching policy $\pi_{0.5}$ [36] on the same episodes; as a separately pretrained model family it places our absolute numbers on an external scale rather than serving as a matched control.

Subgoal frames	AVG
3	98.1
5	98.2
8	98.3
10 (default)	**98.8**

Table 9 Subgoal granularity. Best in **bold**.

C Additional Analysis

C.1 Demonstration Structure

Subgoal granularity (Table 9) is nearly flat: three keyframes already reach 98.1 AVG, within 0.7 of the full ten (98.8). Denser sampling adds frames but no new sub-goals, and it is the sub-goal decomposition the policy consumes, so returns saturate as soon as the plan is complete rather than as soon as the trajectory is densely covered. This is also what keeps the prefix short enough to cache.

D Discussion and Limitations

Language as a Cross-Embodiment Bridge. Under matched observations, real-robot, human-hand, and XR-retargeted demonstrations produce closely matched action predictions (Table 4). The hardware results also use demonstrations sampled from the merged three-source pool rather than from the robot source alone. Together, these results indicate that the structured representation reduces sensitivity to source-specific appearance and coordinates. They do not establish closed-loop invariance across pinned sources, which requires a larger source-controlled rollout study.

Decoupling Reasoning from Latency. Autoregressive language generation is incompatible with the latency budget of real-time control. StellaVLA instead trains the backbone under joint action and spatial-language supervision, then removes the spatial-language expert and caches the demonstration prefix at deployment. Control therefore incurs no autoregressive language-decoding overhead (Table 8).

Limitations and Future Work. Despite its strong performance, our framework has several limitations. *Long-*

horizon compositionality: while *StellaVLA* substantially improves task-level generalization and robustness, success rates on extremely long-horizon, multi-stage tasks (such as the Long Horizon suite in VLA-Arena) remain low for all evaluated models. The demonstration prefix is fixed for the duration of an episode, so it cannot re-plan once execution has drifted, and in-context conditioning alone therefore does not resolve multi-step error accumulation. *Dependence on retrieval quality:* the effectiveness of our in-context adaptation relies on retrieving a relevant expert demonstration. As the wrong-demonstration ablation shows, conditioning on a mismatched demonstration degrades performance below the no-demonstration baseline, so more robust retrieval under visual and semantic domain shift is a critical next step. Finally, *Coarse kinematic discretization:* our textualized kinematic rationale relies on a quantized representation of spatial movements to fit within the VLM's vocabulary. While this quantization suffices to shape the shared representation, it may introduce discretization errors that limit the precision of fine-grained manipulation. Continuous tokenization or hybrid representation schemes could mitigate this.